

QulP#: Hadamard 변환과 lattice codebook을 이용한 LLM 양자화

QulP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks

Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa (2024)

Weight-only PTQ에서 좌표 변환, block feedback, 8차원 vector code를 결합하는 방법

QuIP#의 범위와 두 결과 경로

구분	수행 단계	학습 여부	결과 해석
공통	Weight-only low-bit compression	원래 model 재학습 없음	Activation precision은 weight-code rate와 별도
QuIP#-NoFT	RHT와 BlockLDLQ, E8P 또는 RVQ	Fine-tuning 없음	순수 quantization 경로
QuIP#-FT	NoFT 뒤 선택 parameter 최적화	Development data로 학습	Training-free 결과가 아님

NoFT와 FT는 같은 이름 아래 있지만 계산 비용과 근거가 다른 변형이다.

앞선 QuIP에서 이어받은 세 가지 기반

기반	QuIP의 역할	QuIP#에서의 위치
Layer proxy	한 linear layer의 출력 오차를 Hessian으로 가중	Block quantization의 최적화 대상
Incoherence	큰 coordinate와 민감 방향의 집중을 완화	RHT로 처리 방식을 교체
Adaptive feedback	이미 반올림한 오차를 뒤 coordinate에 반영	BlockLDLQ로 vector block에 확장

QuIP#는 QuIP의 문제 설정을 유지하면서 변환과 codebook, tuning 경로를 바꾼다.

QuIP#가 추가한 세 단계

- | | | |
|----|-------------|--|
| 01 | RHT | Random sign과 Hadamard로 coordinate energy를 분산한다 |
| ↓ | | |
| 02 | BlockLDLQ | 여러 coordinate를 묶어 codeword를 고르고 오차를 전달한다 |
| ↓ | | |
| 03 | E8P와 RVQ | 8차원 구조 codebook으로 2, 3, 4-bit code를 만든다 |
| ↓ | | |
| 04 | Optional FT | 고정한 quantized weight 주변의 선택 parameter를 보정한다 |

Hadamard 하나가 아니라 transform, vector rounding, 선택적 tuning의 결합이다.

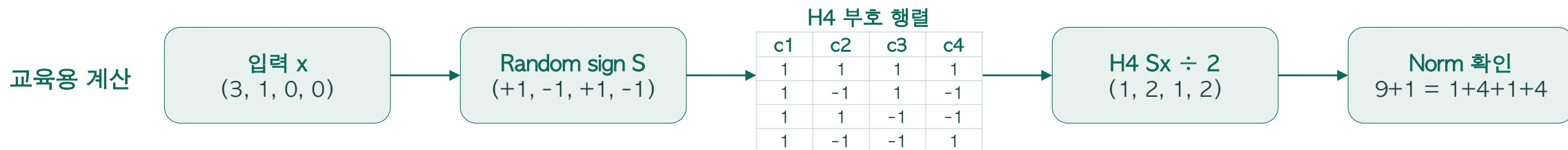
한 layer의 quantization 목표

기호	뜻	해석 경계
W	원래 linear-layer weight	전체 transformer가 아님
W_{hat}	Quantized weight	Code를 decode한 근사 weight
x	Calibration set에서 얻은 layer input	실제 모든 입력 분포가 아님
$H = E\{xx^T\}$	입력으로 만든 proxy Hessian	민감 방향의 가중치
loss	$E\{ (W_{\text{hat}}-W)x ^2\}$	Final task accuracy가 아닌 layer proxy

BlockLDLQ theorem은 이 layer proxy를 다루며 최종 perplexity를 직접 보장하지 않는다.

교육용 4차원 RHT 계산

실제 model transform이 아닌 norm 보존과 coordinate 분산 예시



Orthogonal Hadamard transform은 norm을 보존하면서 energy를 여러 coordinate로 옮긴다.

RHT와 추론의 대응 관계

NoFT의 binary sign 기준으로 quantization 때 바꾼 basis를 activation 경로에서도 맞춘다

01	Input activation x	원래 linear layer 입력
↓		
02	Right RHT	S_V 의 random sign 뒤 Hadamard를 적용
↓		
03	Decoded multiply	변환된 quantized weight code와 곱함
↓		
04	Left RHT	S_U 의 random sign과 Hadamard를 대응 적용
↓		
05	Output y	원래 Wx 를 근사하는 layer 출력

NoFT의 binary signs에서는 matched RHT가 model-preserving basis change다.

Lemma 3.1의 보장 범위

항목	정확한 내용	포함되지 않는 결론
입력	PSD Hessian H와 weight W	특정 LLM accuracy 가정
변환	독립적인 ± 1 sign과 orthogonal scaled Hadamard	항상 모든 값이 동일해짐
확률	실패 확률 δ 에 대해 $1-\delta$ 이상	Deterministic guarantee
결론	변환된 H와 W의 incoherence bound	Perplexity 또는 throughput bound
차원 조건	필요한 Hadamard matrix가 존재하는 차원	임의 차원에서 자동 적용

RHT theorem은 coordinate 집중에 대한 확률적 보장이 model 품질 theorem이 아니다.

Table 1 : RHT와 RFFT 비교

QuIP#-NoFT, 2-bit, WikiText2 perplexity, context 4096, 낮을수록 좋음

변환	Llama 2 7B	Llama 2 13B	Llama 2 70B
RHT	8.22	6.06	4.16
RFFT	8.30	6.08	4.17

이 표에서는 RHT가 낮고, RFFT는 조건에 따라 쓸 수 있는 fallback이다.

Scalar quantization과 vector quantization의 표현 단위

방식	Code 선택	표현 가능한 모양	비용
Scalar quantization	각 weight를 독립적으로 반올림	여러 축의 grid와 hypercube cell	작은 table과 단순 decode
Vector quantization	g개 weight에 codeword 하나	분포에 맞춘 joint codeword	Codebook가 dimension에 따라 커짐

RHT 뒤 ball-shaped 값에는 joint vector code가 더 잘 맞을 수 있다.

Vector quantization의 code rate

- | | | |
|----|--------------------|--|
| 01 | g-coordinate block | 서로 이웃한 g개 weight를 하나의 vector로 묶는다 |
| ↓ | | |
| 02 | k bits per scalar | 한 block index는 k 곱하기 g bits를 사용한다 |
| ↓ | | |
| 03 | Codebook search | $2^{(k \cdot g)}$ 개 후보 중 가까운 codeword를 고른다 |
| ↓ | | |
| 04 | Stored index | Weight 대신 선택한 codeword index를 저장한다 |

2 bits per weight라는 rate와 scalar 2-bit rounding은 같은 메커니즘이 아니다.

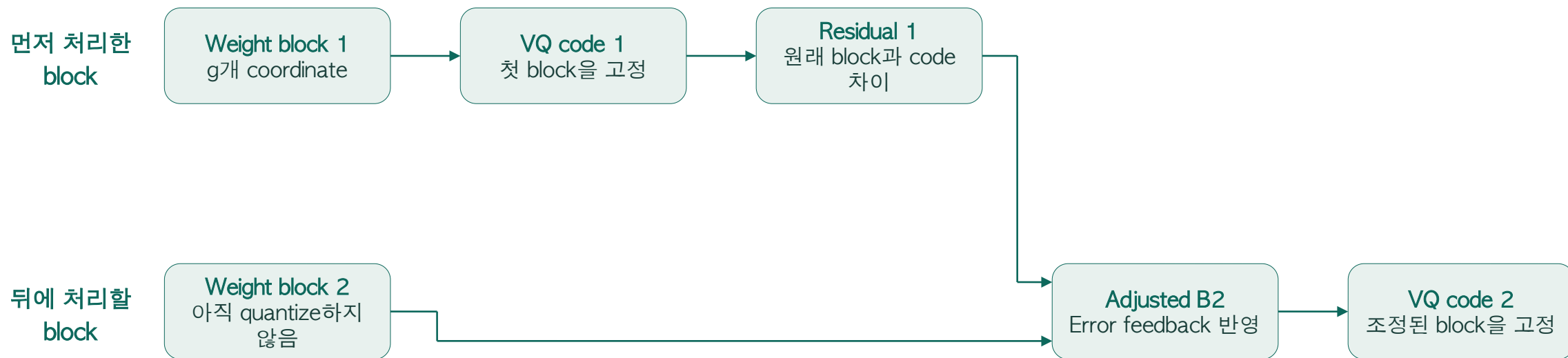
교육용 2차원 VQ 예시

E8P가 아닌 joint-code 선택 원리만 보이는 계산

방식	입력 v	선택 code	Squared error
1-bit scalar grid $\{0,1\}$	(0.49, 0.49)	(0, 0)	$0.49^2 + 0.49^2 = 0.4802$
Custom 2D VQ	(0.49, 0.49)	(0.50, 0.50)	$0.01^2 + 0.01^2 = 0.0002$

이 예시는 joint codeword의 직관만 보이며 E8P 결과가 아니다.

BlockLDLQ의 앞방향 error feedback



고정한 block의 error만 뒤 block의 rounding 입력에 전달한다.

Theorem 4.1의 조건과 결론

구분	원문 조건 또는 결론	의미
Matrix	W 는 $m \times n$, block size g 가 n 을 나눔	g 개 column씩 처리
Sensitivity	H 가 μ -incoherent	민감 방향이 coordinate에 집중되지 않음
Quantizer noise	Error covariance가 $\sigma^2 I$ 이하	VQ error의 second moment 조건
Bound	$\text{Expected proxy} \leq g m \mu^2 \sigma^2 \text{tr}(H^{(1/2)})^2 \div n$	Layer-local expected reconstruction bound
비교	Independent blocks는 $g m \sigma^2 \text{tr}(H)$	Feedback bound와 비교하는 기준

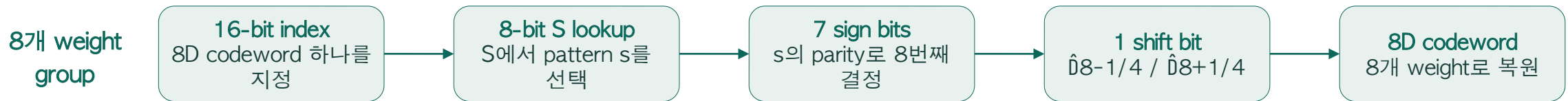
Theorem은 expected layer proxy를 다루며 downstream accuracy를 보장하지 않는다.

E8P의 실제 표현 단위

항목	E8P 값	해석
Vector dimension	8	Weight 8개를 한 group으로 처리
Rate	2 bits per scalar	한 group index는 16 bits
Conceptual entries	2^{16}	가능한 8D codeword 수
Source lookup	$2^8 \times 8$	구조와 symmetry로 작은 table 사용
Representation	1 KiB	FP16 $2^{16} \times 8$ table을 저장하지 않음

E8P는 8개 weight를 16-bit index로 묶는 vector code다.

16-bit E8P index의 구조적 decode



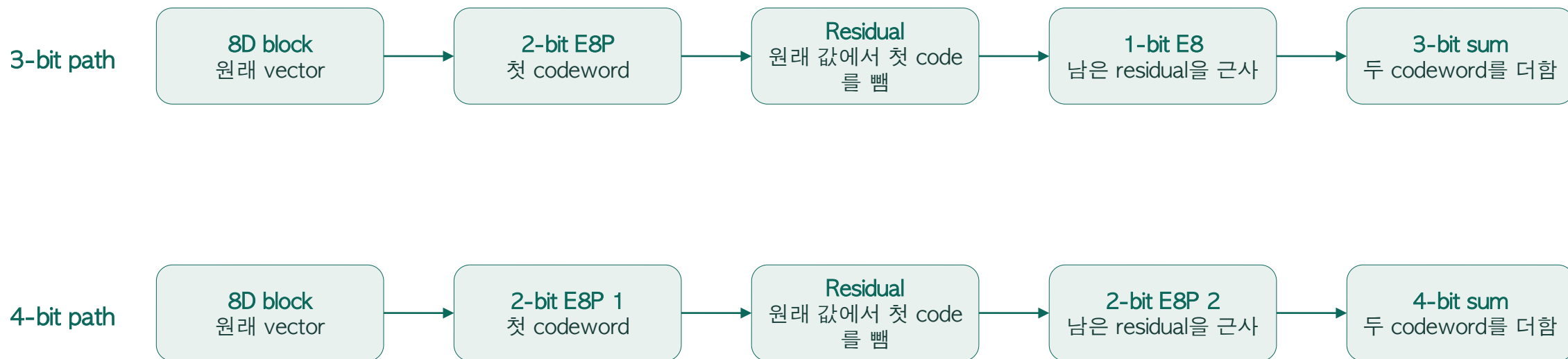
큰 8D codebook를 전부 저장하지 않고 pattern, sign symmetry, shift로 복원한다.

교육용 toy와 E8P를 구분하는 기준

구분	Dimension	Code 구조	말할 수 있는 것
2D toy	2	세 점을 임의로 배치	Joint codeword의 직관
Generic VQ	g	2^g 개 codeword	Rate와 codebook-size 관계
E8P	8	E8 lattice symmetry와 padding	16-bit index, 1 KiB decode 구조

Toy error 차이를 E8P 성능이나 동일-rate benchmark로 해석하면 안 된다.

Residual VQ로 3-bit와 4-bit를 구성한다



두 번째 stage는 첫 stage가 남긴 residual을 처리한다.

QuIP#-NoFT pipeline

01	W와 Hessian H	Pretrained linear layer와 calibration sensitivity를 입력
↓		
02	IP-RHT	W와 H를 변환하고 sign vector S_U, S_V를 보존
↓		
03	BlockLDLQ	g-coordinate block을 feedback과 함께 quantize
↓		
04	E8P 또는 RVQ	2, 3, 4-bit vector index로 weight를 표현
↓		
05	Compressed layer	Code와 matched transform으로 inference 수행

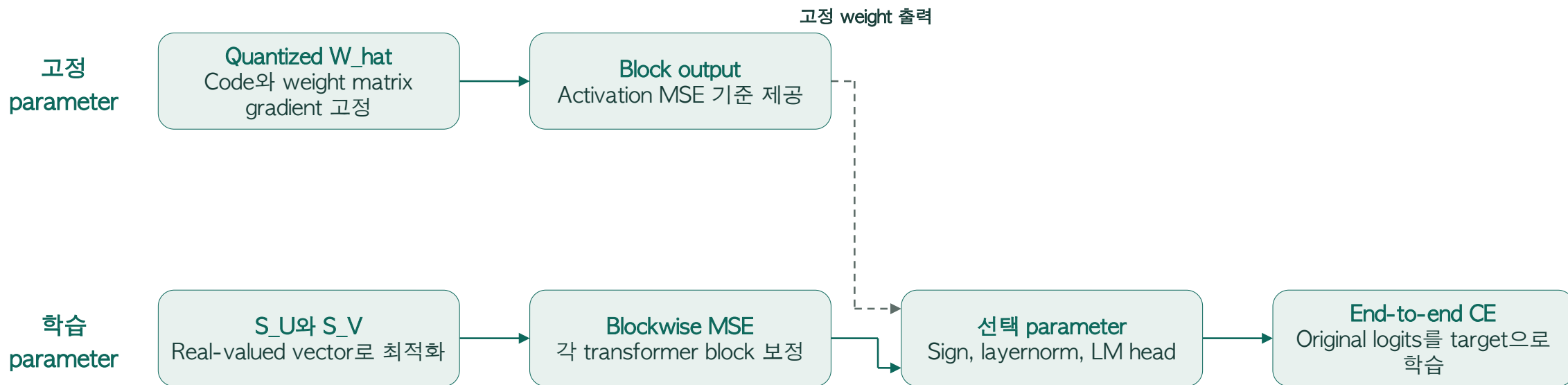
NoFT는 calibration Hessian을 쓰지만 parameter tuning은 하지 않는다.

Layer proxy 뒤에 fine-tuning을 두는 이유

단계	최적화 대상	남는 문제
Layer quantization	한 linear layer의 Hessian-weighted output error	뒤 layer까지 누적되는 mismatch
Blockwise tuning	한 transformer block의 activation MSE	전체 model logit mismatch
End-to-end tuning	Original model logit과의 cross-entropy	Development-set 조건과 compute cost

FT는 layer 사이의 누적 mismatch를 data로 줄인다.

QuIP#-FT에서 고정되는 것과 학습되는 것



Quantized weight는 고정하고 selected parameter는 학습한다.

Fine-tuning의 data와 compute 조건

항목	NoFT	FT
Llama 2 70B quantization	10 GPU-hours 미만	약 100 GPU-hours
Hessian generation	두 시간 모두 제외	두 시간 모두 제외
Blockwise data	해당 없음	Train 256, validation 128 sequences
Blockwise optimization	해당 없음	Adam, 5 epochs, 160 steps, batch 8
Hardware	NVIDIA A100 실험	NVIDIA A100 실험

FT의 quality gain에는 NoFT보다 큰 data와 optimization cost가 따른다.

Table 4: Llama 2 70B의 2-bit ablation

Context 4096, perplexity는 낮을수록 좋음

Variant	WikiText2 PPL	C4 PPL	포함된 구성
QuIP#-FT	3.91	5.71	RHT, E8P, fine-tuning
QuIP#-NoFT	4.16	6.01	RHT와 E8P
NoFT-noE8	4.58	6.51	RHT와 1D half-integer grid

이 고정 protocol 안에서는 E8 codebook과 FT가 각각 추가 개선과 함께 나타난다.

Table 4: Llama 2 7B에서 더 큰 2-bit 격차

Context 4096, perplexity는 낮을수록 좋음

Variant	WikiText2 PPL	C4 PPL	70B와 같은 protocol인가
QulP#-FT	6.19	8.16	같은 Table 4와 context
QulP#-NoFT	8.22	11.0	같은 Table 4와 context
NoFT-noE8	11.2	14.5	같은 Table 4와 context

7B에서도 FT와 lattice codebook의 이점이 실험적으로 나타난다.

Table 2와 Table 4의 context 조건은 다르다

표	Context length	주요 비교	안전한 사용
Table 2	2048	AWQ, OmniQuant, QulP# 포함	2048 protocol 안에서 비교
Table 4	4096	FP16, QulP, AQLM, QulP#와 ablation	4096 protocol 안에서 비교
Appendix Hessian	Llama 2 native 4096	6144 RedPajama sequences	Calibration 조건으로 별도 기록

서로 다른 context의 PPL 숫자를 한 ranking처럼 직접 비교할 수 없다.

Table 5: RTX 4090 proof-of-concept inference

FlashAttention Llama implementation, generation throughput와 peak memory bandwidth 이용률

Model	Bit rate	Tokens/s	Memory bandwidth
Llama 2 7B	2-bit	170.50	29.60%
Llama 2 13B	2-bit	104.83	33.80%
Llama 1 30B	2-bit	51.60	38.39%
Llama 2 70B	2-bit	32.74	56.84%

70B의 56.84% 수치는 named GPU와 implementation에서 얻은 시스템 결과다.

2 bits per weight와 전체 배포 memory의 차이

구성	분류	값·근거	해석 경계
E8P indices	Paper accounting	16 bits per 8 weights	핵심 2-bit code rate
E8P source table	Paper accounting	Model 전체에 공유되는 1 KiB	작지만 0은 아님
Sign vectors	Paper accounting	큰 Llama matrix에서 0.01 bits/weight 미만	FT에서는 FP16 저장
Scales와 retained state	일반 deployment 고려	Appendix F.1 산정 대상 아님	Paper의 정량 claim이 아님
Runtime activation·cache	일반 deployment 고려	Appendix F.1 산정 대상 아님	Weight code rate와 별도

2-bit code rate와 전체 배포 memory는 같은 수치가 아니다.

Formal guarantee와 model evidence의 경계

근거	성립하는 결론	성립하지 않는 결론
Lemma 3.1	RHT 뒤 incoherence의 high-probability bound	Final PPL 개선 보장
Theorem 4.1	조건부 expected layer proxy bound	전체 LLM accuracy 보장
Tables 1과 4	지정 model-context의 PPL 비교	모든 dataset의 ranking
Table 5	RTX 4090 implementation의 throughput	Hardware-independent speedup
Appendix F	특정 calibration과 tuning cost	모든 model의 최적 recipe

Theory와 model-system experiment는 서로 다른 범위를 검증한다.

QuIP#의 전체 논리

01	Coordinate 준비	NoFT는 binary sign과 Hadamard로 coordinate 집중을 완화
↓		
02	Block quantization	Vector code를 고르고 앞 block residual을 뒤 block에 반영
↓		
03	Structured code	E8P와 RVQ로 8D low-bit index를 compact하게 decode
↓		
04	선택적 FT	NoFT를 유지하거나 selected parameter FT를 추가
↓		
05	조건부 검증	Matched context의 품질표와 named GPU의 inference로 평가

QuIP#는 변환, feedback, code 구조를 함께 설계한다.